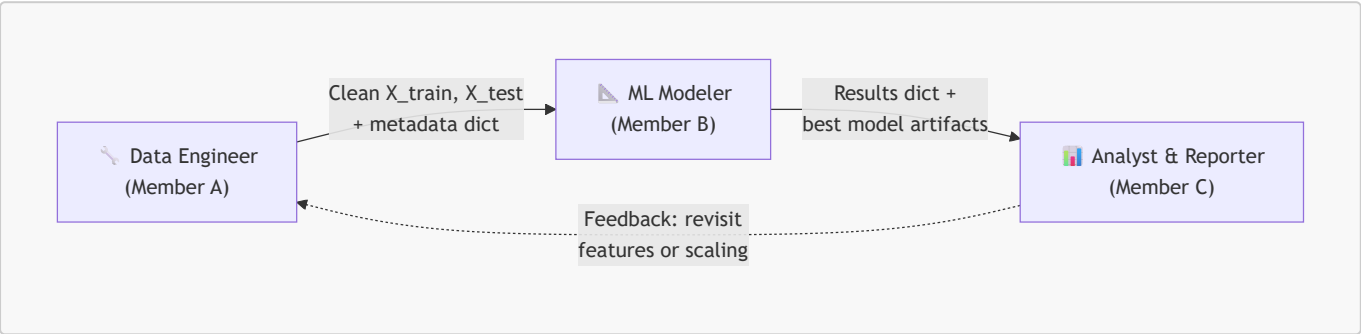


Part 2 — Team Structure & Work Division

Project: Data Fitting on Melbourne Housing Snapshot **Team Size:** 3 members **Architecture:**
`data_pipeline.py` → `model_comparison.py` → `advanced_methods.py` + `part2_notebook.ipynb`

The 3 Personas



Persona 1: Data Engineer (Member A)

Ownership: `data_pipeline.py`, `part2/data/` **Core Responsibility:** Deliver a clean, leakage-free, model-ready dataset.

Tasks

#	Task	Output
A1	Load Melbourne Housing CSV, perform initial EDA (shape, dtypes, distributions)	EDA summary printed in notebook
A2	Implement <code>DataPipeline.fit_transform()</code> : handle missing values in <code>BuildingArea</code> (~47%) and <code>YearBuilt</code> (~40%) using a chosen imputation strategy	Fitted pipeline object with stored <code>self.imputation_values</code>
A3	Implement one-hot encoding for categorical columns (e.g., <code>Type</code> , <code>Regionname</code>). Store the resulting column list in <code>self.encoded_columns</code>	Consistent column set across train/test
A4	Implement feature scaling (standardization). Store <code>self.scalers</code> (mean, std per column)	Scaled <code>X_train</code> , <code>X_test</code> as NumPy arrays
A5	Implement <code>DataPipeline.transform()</code> for the test set using only the fitted state	Transformed <code>X_test</code> with zero data leakage
A6	Split data using <code>train_test_split()</code> (80/20, fixed <code>random_state</code>)	<code>X_train</code> , <code>X_test</code> , <code>y_train</code> , <code>y_test</code> + <code>metadata dict</code>

Deliverables (Handover to Member B)

```
# The "Contract" from A → B
X_train: np.ndarray      # shape (n_train, p), scaled, no NaNs
X_test:  np.ndarray      # shape (n_test, p), scaled, no NaNs
y_train: np.ndarray      # shape (n_train,)
y_test:  np.ndarray      # shape (n_test,)
metadata: dict           # {
                        #   'feature_names': [...],
                        #   'target_name': 'Price',
                        #   'pipeline': fitted DataPipeline instance,
                        #   'imputation_strategy': 'median',
                        #   'scaling_method': 'standardize'
                        # }
```

Persona 2: ML Modeler (Member B)

Ownership: `model_comparison.py`, `cross_validation.py` (from Part 1) **Core Responsibility:** Train all models, tune hyperparameters, and produce a ranked comparison.

Tasks

#	Task	Output
B1	Implement <code>train_models()</code> : run our custom <code>ols_fit(X, y)</code> from Part 1 on the clean data	<code>beta_hat_ols, y_pred_ols</code>
B2	Implement <code>train_models()</code> : run our custom <code>ridge_fit(X, y, lam)</code> from Part 1	<code>beta_hat_ridge, y_pred_ridge</code>
B3	Run <code>sklearn.LinearRegression</code> and <code>sklearn.Ridge</code> as baselines	<code>y_pred_sklearn_ols, y_pred_sklearn_ridge</code>
B4	Implement k-fold cross-validation (<code>k=5</code> or <code>k=10</code>) to tune λ for Ridge/Lasso	<code>best_lambda</code> , CV score curve
B5	Compute MAE, RMSE, R^2 for all models on the test set using <code>model_metrics()</code>	Comparison dict / DataFrame
B6	Implement <code>comparison_table()</code> to format results into a summary DataFrame	Ranked model table
B7	Run <code>evaluate_gauss_markov_assumptions()</code> on the best model's residuals	Breusch-Pagan p-value, VIF table

Deliverables (Handover to Member C)

```
# The "Contract" from B → C
results: dict           # {
                        #   'OLS_custom': {
                        #     'coefficients': np.ndarray,
```

```
        # 'predictions_test': np.ndarray,
        # 'metrics': {'MAE': ..., 'RMSE': ..., 'R2': ..., 'Adj_R2':
...}

        # },
        # 'Ridge_custom_lambda_X': { ... },
        # 'sklearn_OLS': { ... },
        # ...
        # }

best_model_name: str           # e.g., 'Ridge_custom_lambda_0.5'
best_beta: np.ndarray          # Coefficients of the best model
best_residuals: np.ndarray     # y_test - y_pred of the best model
best_lambda: float            # Optimal lambda from CV
cv_scores: dict               # {'lambda_values': [...], 'mean_scores': [...]}
gauss_markov_results: dict     # {'Breusch-Pagan': p_val, 'VIF': df, ...}
comparison_df: pd.DataFrame    # The formatted comparison table
```

Persona 3: Analyst & Reporter (Member C)

Ownership: `part2_notebook.ipynb`, `advanced_methods.py`, report sections

Core Responsibility:
Visualize, interpret, and write the narrative that ties the math to the real-world meaning.

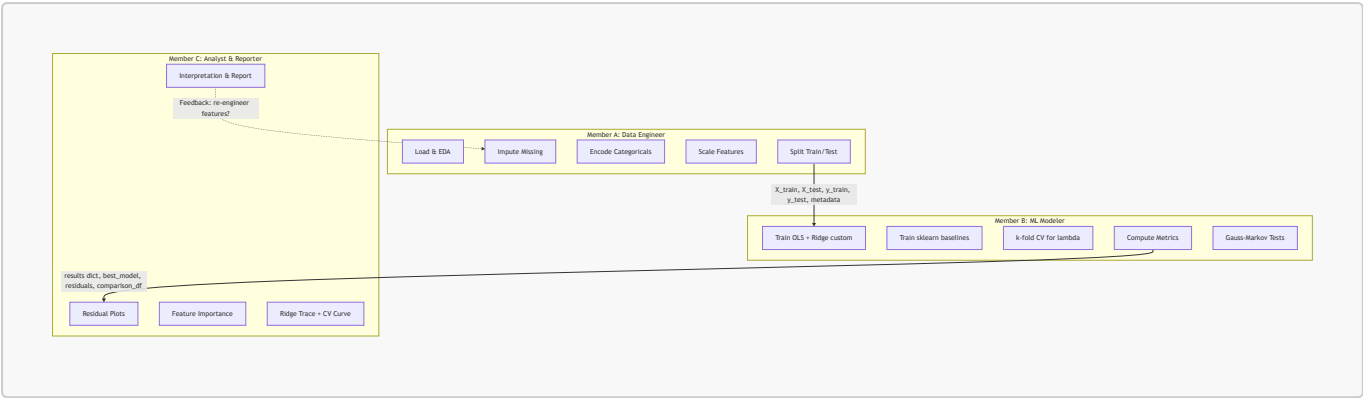
Tasks

#	Task	Output
C1	Plot the 4 residual diagnostic plots for the best model (Residuals vs Fitted, Q-Q, Scale-Location, Residuals vs Leverage)	4-panel figure
C2	Plot standardized regression coefficients as a horizontal bar chart for feature importance	Feature importance figure
C3	Plot the Ridge Trace (λ vs coefficients) and the CV error curve	2 figures
C4	Interpret the comparison table: why did Ridge outperform OLS? What does λ do to multicollinear features like <code>Rooms</code> vs <code>Bedroom2</code> ?	Written analysis paragraphs
C5	Interpret the Gauss-Markov test results: does heteroscedasticity exist? Is the normality assumption violated?	Written discussion
C6	Write the Discussion & Conclusion section: connect the math to the Melbourne housing market context	Final report section
C7	(Bonus) Implement and evaluate Kernel Ridge or Bayesian Linear Regression in <code>advanced_methods.py</code>	Advanced model results

Deliverables (Final Output)

- Completed `part2_notebook.ipynb` with all figures, tables, and narrative
- Contribution to the LaTeX report in `report/`

Contracts Summary



Contract	From	To	Exact Artifact
Contract 1	A → B	Data Engineer → ML Modeler	<code>X_train</code> , <code>X_test</code> , <code>y_train</code> , <code>y_test</code> (NumPy arrays) + <code>metadata</code> dict containing feature names, pipeline instance, and preprocessing choices
Contract 2	B → C	ML Modeler → Analyst	<code>results</code> dict (all models' coefficients, predictions, metrics), <code>best_residuals</code> , <code>best_beta</code> , <code>best_lambda</code> , <code>cv_scores</code> , <code>gauss_markov_results</code> , <code>comparison_df</code>
Contract 3	C → A	Analyst → Data Engineer	Feedback loop: if residual analysis reveals issues (e.g., heteroscedasticity suggests log-transforming <code>Price</code>), request a pipeline re-run

Sync Points (Mandatory Joint Decisions)

Sync Point 1: Imputation & Feature Engineering Review

When: After Member A completes tasks A1–A3 (EDA + imputation + encoding) **Who:** All 3 members

Decision: Agree on the imputation strategy for `BuildingArea` (47% missing) and `YearBuilt` (40% missing). Options: median fill, KNN imputer, or drop rows. Also decide whether to engineer new features (e.g., `Age = 2026 - YearBuilt`). This decision directly impacts the design matrix `XX` that Member B will model.

[!IMPORTANT] This is the single most impactful decision in the project. If imputation is poor, every downstream model and metric is compromised.

Sync Point 2: Multicollinearity & Feature Selection Gate

When: After Member B runs VIF on the initial model (task B7, first pass) **Who:** All 3 members **Decision:**

Review the VIF table. If `Rooms` and `Bedroom2` both have `VIF > 10` (as our earlier synthetic test predicted), the team must jointly decide which to drop. This requires Member A to re-run the pipeline with the dropped column, and Member C to understand the interpretive consequences.

Sync Point 3: Best Model Selection & Report Alignment

When: After Member B completes the comparison table (task B6) and Member C has draft residual plots (task C1) **Who:** All 3 members **Decision:** Officially select the "best model" for the report. Is it the

model with the lowest RMSE? Or should we prefer a simpler model (OLS) if its R^2 is only marginally worse than Ridge? Also align on the narrative: what story does the report tell?

Suggested Timeline

